cura_setting_sweep.py

Automatisierter Cura-Setting-Sweep

Bedienungsanleitung · Stand Juni 2026

Dieses Programm testet automatisch alle Print-Settings von UltiMaker Cura (Creality Ender-3, PLA, 0.4 mm Düse, Standard Quality) daraufhin, ob sie einen messbaren Einfluss auf den erzeugten G-Code haben – und damit potenziell sichtbare optische Auswirkungen auf den 3D-Druck.

Getestet werden: numerische Settings (Float/Int) per Verdreifachung des Default-Werts **und** Dropdown-Settings (Enum) durch Ausprobieren jeder verfügbaren Option. Bedingte Settings, die nur bei bestimmten Voreinstellungen aktiv sind, werden mit den passenden Vorbedingungen getestet.

---

1. Schnellstart

Voraussetzungen

Komponente	Anforderung
Python	3.9 oder neuer
UltiMaker Cura	5.x installiert (CuraEngine wird mitgeliefert)
openpyxl	wird beim ersten Start automatisch installiert
STL-Datei	im Projektordner (z.B. cube.stl)

Aufruf

```
python3 cura_setting_sweep.py
```

Das Programm findet Modell und CuraEngine automatisch. Alle Ergebnisse landen im Projektordner als .xlsx und .csv.

Kommandozeilen-Optionen

Option	Bedeutung	Beispiel
--limit N	Nur die ersten N Settings testen (Test-Modus)	--limit 10
--only KEY,KEY	Nur bestimmte Settings (kommagetrennt)	--only layer_height,infill_sparse_density
--list	Settings auflisten ohne zu slicen	--list
--calib N	Anzahl Baseline-Kalibrierläufe (Default: 5)	--calib 8
--no-cascade	Formel-Kaskadierung deaktivieren	--no-cascade

---

2. Funktionsweise

Überblick

Das Programm arbeitet in drei Phasen:

Phase	Was passiert
1. Kalibrierung	Die Baseline wird mehrfach gesliced (Standard: 5×). Das Rauschband der Metriken wird gemessen.
2. Sweep	Jedes Setting wird einzeln getestet: Wert ändern → slicen → Metriken vergleichen.
3. Export	Ergebnisse werden in Excel (.xlsx) und CSV gespeichert.

Warum kein Byte-Vergleich?

CuraEngine erzeugt bei identischen Eingaben leicht unterschiedlichen G-Code (nicht-deterministisches Verhalten bei Reise- und Combing-Bewegungen). Ein direkter Dateivergleich würde daher überall faelschlich "Veraenderung" melden.

Stattdessen werden robuste, reihenfolge-unabhängige **Metriken** extrahiert und verglichen. Das Rauschband wird durch die Kalibrierläufe automatisch bestimmt.

Gemessene Metriken

Metrik	Beschreibung	Floor (Mindestschwelle)
Layer-Anzahl	Anzahl der Layer im G-Code	0.5 (d.h. ±1 Layer)
Filament	Extrudiertes Filament in mm	0.05 mm
Extr.-Distanz	Summe aller Extrusions-Wegstrecken	5.0 mm
Extr.-Bewegungen	Anzahl der Extrusions-Moves	20 Moves
Seam-X	Mittl. X-Position des Nahtpunkts	10.0 mm
Seam-Y	Mittl. Y-Position des Nahtpunkts	10.0 mm

> *Nicht-visuelle Größen (Druckzeit, Temperatur, Lüfter) werden bewusst nicht berücksichtigt – sie ändern die Druckbedingungen, nicht die Form.*
>
> *Seam-X und Seam-Y messen die mittlere Startposition der äußeren Wandextrusion pro Layer (;TYPE:WALL-OUTER Abschnitt). Damit wird z.B. die Einstellung z_seam_x / z_seam_y korrekt erkannt.*

Verifikation von Treffern

Da CuraEngine nicht-deterministisch ist, wird jeder "Veraenderung"-Treffer nochmals mit 2 zusaetzlichen Slices ueberprueft. Nur wenn die Abweichung reproduzierbar ueber der Toleranz bleibt, wird "Veraenderung" endgueltig gemeldet. Das eliminiert Rausch-Fehltreffer bei komplexen Modellen.

---

3. Setting-Typen und Testlogik

Numerische Settings (Float / Int)

Pro Setting wird der Default-Wert vom CuraEngine aufgelöst (inkl. Standard-Quality-Overrides). Dann wird ein Testwert berechnet:

Situation	Testwert
Default > 0	3 × Default (Verdreifachung)
Default = 0 (Float)	Ersatzwert 1.0 (da 3×0 = 0 nicht testbar)
Default = 0 (Int)	Ersatzwert 1

Dropdown-Settings (Enum)

Für jede Dropdown-Liste werden alle **Nicht-Standard-Optionen** der Reihe nach getestet. Das Programm stoppt beim ersten Fund einer Veränderung.

Beispiel: `z_seam_type` hat die Optionen *back, shortest, random, sharpest_corner*. Standard ist *back*. Getestet werden also: *shortest*, dann *random*, dann *sharpest_corner* – bis die erste Veränderung gefunden wird.

Im Excel-Ergebnis steht in der Spalte **Testwert**, welche Optionen getestet wurden und welche die Veränderung ausgelöst hat (erste Option mit *)

Formel-Kaskadierung

Viele Cura-Settings haben Formeln als Default-Wert, z.B.:

```
infill_line_width = line_width × 1.0
```

Wenn ein übergeordnetes Setting geändert wird, werden alle abhängigen Settings automatisch neu berechnet und mit übergeben. Das stellt sicher, dass der Test den gleichen Zustand erzeugt wie die manuelle Einstellung in Cura.

Der Abhängigkeitsgraph wird per BFS (Breadth-First Search) über alle fdmprinter.def.json-Einträge aufgebaut und beim Programmstart einmalig geladen.

> *--no-cascade deaktiviert die Kaskadierung (altes Verhalten, nur für Debugging).*

Bedingte Settings (enabled-Ausdrücke)

Viele Cura-Settings sind nur sichtbar, wenn andere Settings bestimmte Werte haben – z.B. ist `z_seam_x` nur aktiv, wenn `z_seam_type == 'user_specified'`. In Cura erscheinen solche Settings erst, wenn man die Checkbox anklickt oder eine Dropdown-Option wählt.

Das Programm analysiert automatisch den **enabled-Ausdruck** jedes Settings und extrahiert die notwendigen Vorbedingungen:

Muster im Ausdruck	Was passiert
key == 'wert'	Enum-Setting auf diesen Wert setzen
key > 0	Numerisches Setting auf 1 setzen (falls Default 0)
bool_flag (solo)	Boolean-Setting auf true setzen

Für jede eindeutige Kombination von Vorbedingungen wird eine eigene **Feature-Baseline** berechnet (mit denselben Vorbedingungen), damit der Vergleich fair bleibt. Diese Baselines werden gecacht – identische Vorbedingungssets (z.B. alle Support-Settings teilen *support_enable=true*) werden nur einmal berechnet.

Permanent deaktivierte Settings

Settings mit `enabled: False` (unveränderlich, nicht durch andere Settings beeinflussbar) werden als "Permanent deaktiviert" markiert und nicht getestet. In der aktuellen Cura-Version sind das ca. 23 Settings.

Ausgeschlossene Kategorien

Die Kategorien **machine_settings** (Drucker-Hardware-Parameter) und **command_line_settings** (interne CuraEngine-Flags, die nicht als Print-Settings gedacht sind) werden nicht getestet.

---

4. Excel-Ausgabe

Spalten

Spalte	Inhalt
Kategorie	Cura-Kategorie (z.B. shell, infill, support)
Setting Key	Interner Name des Settings
Label	Anzeigename in Cura
Typ	float, int oder enum
Default-Wert	Von CuraEngine aufgelöster Default-Wert
Testwert	Getesteter Wert (3× default oder enum-Optionen)
Status	Ergebnis (s. Farblegende unten)
Geänderte Metriken	Welche Metriken haben sich verändert?
Kaskadiert	Anzahl abhängiger Settings, die mit übergeben wurden
Vorbedingungen	Gesetzte Vorbedingungen für bedingte Settings
G-Code-Datei	Pfad zum erzeugten Test-G-Code
Dauer (s)	Slice-Dauer in Sekunden

Farblegende

Farbe	Status	Bedeutung
Grün	Veränderung	Setting hat messbaren Einfluss auf den G-Code
Hellgrau	keine Veränderung	Setting hat keinen Einfluss (innerhalb Toleranz)
Gelb	nicht testbar (Default 0)	Default ist 0 und Setting ist Float/Int ohne Alternativwert
Rot	Fehler/Timeout	CuraEngine-Fehler oder Timeout beim Slicen
Dunkelgrau	Permanent deaktiviert	enabled=False, nie testbar

Zweites Tabellenblatt: Baselines

Das zweite Excel-Blatt zeigt die kalibrierten Referenzwerte und Toleranzen der Haupt-Baseline. Nützlich zur Diagnose, wie stabil das Rauschband ist.

---

5. Interne Architektur

Funktion	Aufgabe
collect_settings()	Liest fdmprinter.def.json und liefert alle testbaren Settings (float, int, enum) mit Typ, Default, enable
collect_all_settings()	Liest ALLE Settings inkl. Bool/String (für Kaskadierung und enabled-Auswertung)
build_dep_graph()	Baut BFS-Abhängigkeitsgraph: welches Setting hängt von welchem ab?
cascade_overrides()	Berechnet alle abhängigen Settings neu, wenn ein Eltern-Setting geändert wird
build_enable_ns()	Erstellt Auswerte-Namespace für enabled-Ausdrücke (Python eval)
get_preconditions()	Analysiert enabled-Ausdruck und extrahiert notwendige Vorbedingungen
run_slice()	Ruft CuraEngine headless auf, gibt (ok, stdout, dauer) zurück
extract_features()	Liest G-Code und extrahiert alle Metriken (Layer, Filament, Seam…)
calibrate()	Berechnet Referenz + Toleranz aus mehreren Baseline-Messungen
compare()	Prüft ob Testmetriken außerhalb der Toleranz liegen
verify_change()	Wiederholt Slice 2× zur Bestätigung eines Treffers
write_excel()	Schreibt Ergebnis-xlsx mit Farben und Formatierung

---

6. Konfiguration

Konstanten im Skript

Konstante	Default	Bedeutung
CALIB_RUNS	5	Baseline-Kalibrierläufe
VERIFY_RUNS	2	Verifikationsläufe pro Treffer (0 = aus)
SLICE_TIMEOUT	120 s	Max. Slice-Dauer (Sekunden)
SUBSTITUTE_FLOAT	1.0	Testwert wenn Default = 0.0 (Float)
SUBSTITUTE_INT	1	Testwert wenn Default = 0 (Int)
DECISION_FLOORS	s.o.	Mindest-Schwellwerte pro Metrik
FIXUP_OVERRIDES	s.o.	CuraEngine-5.13.0-macOS-Pflichtfelder
SKIP_PRECONDITIONS	s.o.	Felder, die nicht per CLI gesetzt werden können

Pfade

CuraEngine und Ressourcen werden automatisch für macOS, Windows und Linux erkannt. Das Modell wird im Projektordner gesucht (*.stl). G-Codes werden in `gcode_output/` gespeichert.

---

7. Häufige Probleme

Problem	Lösung
CuraEngine nicht gefunden	Cura korrekt installiert? Pfad in Skript-Konstanten prüfen.
Kein STL-Modell gefunden	cube.stl (oder anderes .stl) in den Projektordner legen.
Sehr lange Laufzeit	Mit --limit 10 testen. Bei Support-Settings ist eine Feature-Baseline nötig (extra Kalibrierung).
openpyxl fehlt	Wird automatisch installiert – einmal kurz warten.

'keine Veränderung' obwohl in Cura sichtbar	Möglicherweise ist die Metrik-Schwelle zu hoch. DECISION_FLOORS im Skript anpassen.
Enum-Setting wird nicht getestet	Prüfen ob das Setting eine enabled-Bedingung hat, die nicht aufgelöst werden konnte. Im Log nach

---

8. Ablauf (vereinfacht)

```
START
  ■■ Abhängigkeitsgraph laden (fdmprinter.def.json)
  ■■ Settings sammeln (float, int, enum)
  ■■ Haupt-Baseline × 5 slicen → ref + tol kalibrieren
  ■■ FÜR JEDES SETTING:
       ■■ Vorbedingungen analysieren (enabled-Ausdruck)
       ■    ■■ Permanent deaktiviert? → 'Permanent deaktiviert'
       ■    ■■ Vorbedingungen nötig?  → Feature-Baseline (gecacht)
       ■■ ENUM: jede Option testen bis erste Veränderung
       ■■ FLOAT/INT: Default × 3 (oder Ersatzwert) testen
       ■    ■■ Abhängige Settings kaskadieren
       ■■ Vergleich mit (Feature-)Baseline
       ■■ Bei Treffer: 2× Verifikation
  ■■ Excel + CSV schreiben
END
```